

Báo cáo bài tập EX11.1:

convolutional_network

Some configs

- Chúng ta sẽ setup một số hyper-parameters cũng như một số giá trị cần dùng theo hướng dẫn nhé
- Ở đây, mình muốn các bạn sử dụng Cuda, hãy vào runtime, rồi change the runtime type sang GPU nhé

```
In [1]: # Trước hết, chúng ta import một số thư viện cần thiết đã
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
import numpy as np
import matplotlib.pyplot as plt
import random
```

```
In [2]: # Số classes trong tập MNIST
num_classes = 10

# Số epoch
epochs = 10

# Các tham số cần thiết cho quá trình training.
learning_rate = 0.01
batch_size = 64
display_step = 100

# Path lưu best model
checkpoint = 'best_model_mnist.pth' # có thể để dạng *.pth

# device chúng ta dùng cuda
device = 'cuda' if torch.cuda.is_available() else 'cpu'
#assert device == 'cuda'
```

Dataloader

```
In [3]: # Transform image
transform=transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Load dataset từ torchvision.datasets
train_dataset = datasets.MNIST('../data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('../data', train=False, transform=transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size)
```

Model

- Trong bài này, chúng ta sẽ định nghĩa một class Net, nó sẽ có cấu trúc như hình ở đầu notebook
- Bạn hãy chỉnh các tham số cho phù hợp nhé :)

```
In [4]: class Net(nn.Module):
def __init__(self):

    super(Net, self).__init__()
    self.dropout = nn.Dropout()
    self.relu = nn.ReLU()
    self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1)
    self.maxpool1 = nn.MaxPool2d(2)
    self.conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=1)
    self.maxpool2 = nn.MaxPool2d(2)
    self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1)
    self.maxpool3 = nn.MaxPool2d(2)
    self.fc1 = nn.Linear(128, 64)
    self.fc2 = nn.Linear(64, 32)
    self.fc3 = nn.Linear(32, 10)

def forward(self, x):
    x=self.maxpool1(self.relu(self.conv1(x)))
    x=self.maxpool2(self.relu(self.conv2(x)))
    x=self.maxpool3(self.relu(self.conv3(x)))
    x = x.view(x.size(0), -1)
    x=self.relu(self.fc1(x))
    x=self.relu(self.fc2(x))
    x=self.fc3(x)# bạn hãy xem xét quá trình forward nhé
    return x
```

```
In [5]: # call model, đừng quên set device nhé
model = Net().to(device)

# Load lại pretrained model (nếu có)
try:
    model.load_state_dict(torch.load(checkpoint, map_location=device))
    print(f"--- Đã tải thành công model từ: {checkpoint}")
except:
    print("!!! Hãy train để có checkpoint file")

--- Đã tải thành công model từ: best_model_mnist.pth
```

```

In [6]: criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
best_val_loss = 999

for epoch in range(1,2):
    # Quá trình training
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad() # zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward() # backward()
        optimizer.step() # update
        if batch_idx % display_step == 0:
            print('Train Epoch: {} [{}/{}] ({:.0f}%) \t Train Loss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))

    # Quá trình testing
    model.eval() # tắt hoàn toàn dropout
    test_loss = 0
    correct = 0
    # set no grad cho quá trình testing
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            log_output = F.log_softmax(output, dim=1) # Log softmax dùng F, chú ý dim
            test_loss += F.nll_loss(log_output, target, reduction='sum').item()
            pred = output.argmax(dim=1, keepdim=True) # argmax để lấy predicted label, chú ý dim, và keepdim = True
            correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)
    if test_loss < best_val_loss:
        best_val_loss = test_loss
        torch.save(model.state_dict(), checkpoint) # Lưu lại model
    print("***** TEST_ACC = {}% *****".format(correct))

```

```

Train Epoch: 1 [0/60000 (0%)]    Train Loss: 1.037322
Train Epoch: 1 [6400/60000 (11%)]    Train Loss: 0.680005
Train Epoch: 1 [12800/60000 (21%)]    Train Loss: 0.482362
Train Epoch: 1 [19200/60000 (32%)]    Train Loss: 0.425378
Train Epoch: 1 [25600/60000 (43%)]    Train Loss: 0.336467
Train Epoch: 1 [32000/60000 (53%)]    Train Loss: 0.353239
Train Epoch: 1 [38400/60000 (64%)]    Train Loss: 0.309722
Train Epoch: 1 [44800/60000 (75%)]    Train Loss: 0.498904
Train Epoch: 1 [51200/60000 (85%)]    Train Loss: 0.378255
Train Epoch: 1 [57600/60000 (96%)]    Train Loss: 0.388180
***** TEST_ACC = 9036% *****

```

```
In [7]: # Load lại model đã train
model.load_state_dict(torch.load(checkpoint, map_location=device))
# Set eval phase nhé bạn
model.eval()
```

```
Out[7]: Net(
  (dropout): Dropout(p=0.5, inplace=False)
  (relu): ReLU()
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1))
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1))
  (maxpool3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=128, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (fc3): Linear(in_features=32, out_features=10, bias=True)
)
```

```
In [8]: item = iter(test_loader)
```

```
In [9]: data, target = item.__next__() # Lấy một batch ra
```

```
In [10]: test_idx = random.choice(range(len(data))) # Lấy index của một phần tử của một batch
```

```
In [11]: data = data[test_idx]
target = target[test_idx]
assert data.shape == (1,28,28)
```

```
In [12]: # thử predict

def plot(data,model):
    data = data.unsqueeze(0).to(device) # unsqueeze data, đồng thời set device
    output = model(data)
    output = F.log_softmax(output, dim=1) # Log softmax, chú ý dim
    pred = output.argmax(dim=1, keepdim=True) # argmax, chú ý keepdim
    print("Predict Number : ", pred[0][0].detach().cpu().numpy())
    plt.imshow(data[0][0].detach().cpu().numpy(), cmap='gray')
    plt.show()
```

```
In [13]: plot(data,model)
```

Predict Number : 8

